



Kubernetes Handbook

1. Kubernetes Fundamentals

What is Kubernetes?

- Kubernetes is an **open-source** Container Orchestration Platform.
- It automates deployment, scaling, networking and management of containerized applications.
- Originally developed by **Google**.
- Often referred to as **K8s**.

Kubernetes helps you run containers **at scale** in a **reliable** and **efficient** way.

Why Kubernetes?

Using Docker alone is great for running containers, but managing them in production is hard. Kubernetes solves these problems.

Problems with Docker alone:

- ✗ Manual deployment of containers
- ✗ Manual scaling
- ✗ No self-healing (if a container dies, it stays down)
- ✗ Difficult load balancing
- ✗ Rolling updates and rollbacks are hard
- ✗ No built-in service discovery

Features

- | | |
|---------------------------|-----------------------------|
| ✓ Container Orchestration | ✓ Rolling Updates |
| ✓ Auto Scaling | ✓ Rollbacks |
| ✓ Self Healing | ✓ High Availability |
| ✓ Load Balancing | ✓ Resource Management |
| ✓ Service Discovery | ✓ Declarative Configuration |

Kubernetes Architecture (High Level)



Developer



Docker Image



Kubernetes Cluster



Running Pods

You define the desired state, Kubernetes makes it happen!



Interview Tip

Docker is used to **build** and **run** containers.

Kubernetes is used to **orchestrate**, **scale** and **manage** containers.

Docker **packages** the application, Kubernetes **runs** it in production.

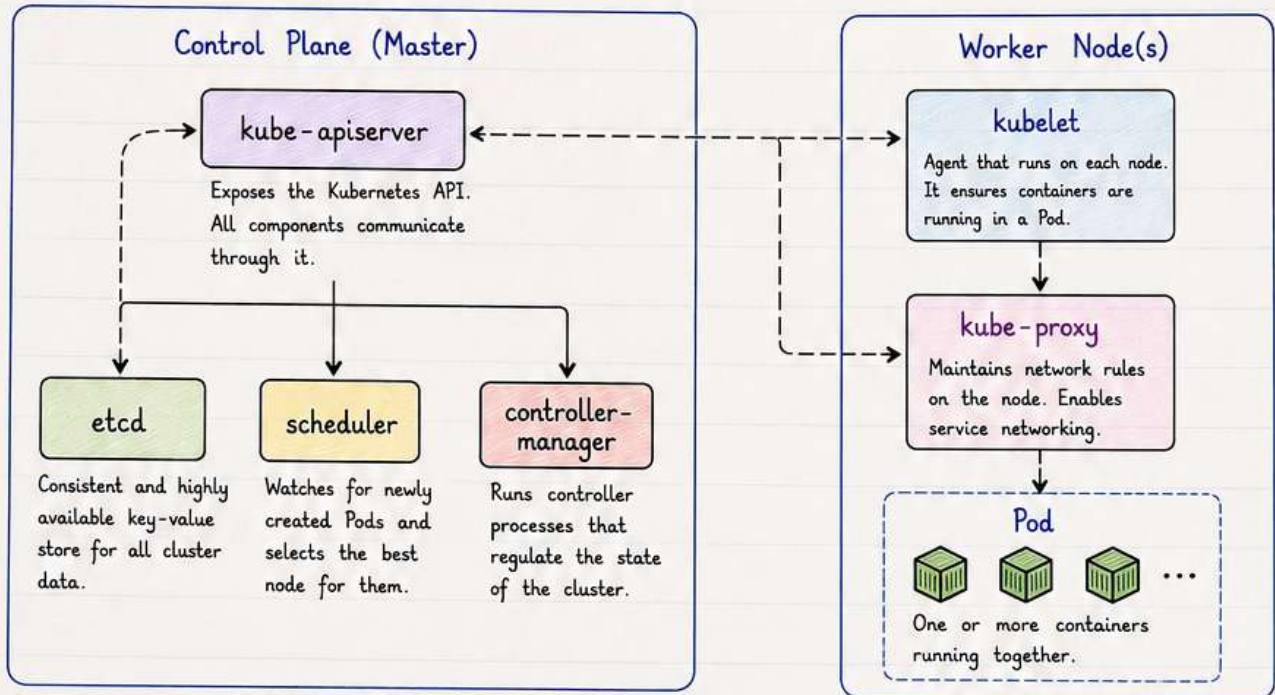




Kubernetes Handbook

2. Kubernetes Architecture

A Kubernetes cluster has a **Control Plane (Master)** that manages the cluster and **Worker Nodes** that run the containerized applications.

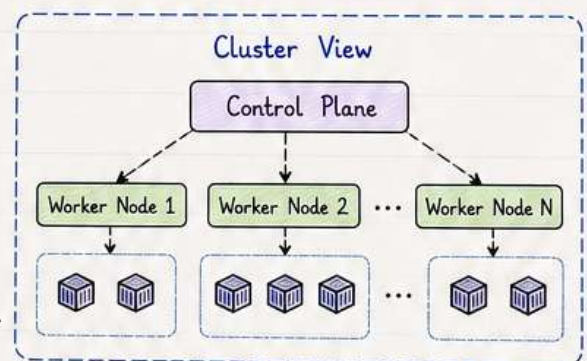


Key Components Explained

- kube-apiserver** - The front-end for the Kubernetes control plane. Validates and processes REST requests and updates the cluster state.
- etcd** - Distributed key-value database used to store all cluster data.
- scheduler** - Assigns Pods to the best suitable node based on resources and constraints.
- controller-manager** - Runs controllers like Node Controller, ReplicaSet Controller, Endpoints Controller to keep the cluster in desired state.
- kubelet** - Ensures that containers in a Pod are running as expected.
- kube-proxy** - Handles networking and ensures traffic is routed to the correct Pods.
- Container Runtime** - Software that runs containers (e.g., containerd, Docker).

How it Works?

1. You create a resource (e.g., Pod, Deployment) using `kubectl`.
2. Request goes to `kube-apiserver`.
3. Scheduler decides which node should run the Pod.
4. Kubelet on that node creates the Pod and starts containers.
5. Kube-proxy ensures the Pod is reachable via network.
6. Controllers continuously monitor and maintain the desired state.



Interview Tip

Kubernetes works on the principle of "Desired State vs Actual State".

The control plane constantly works to make the actual state match the desired state.





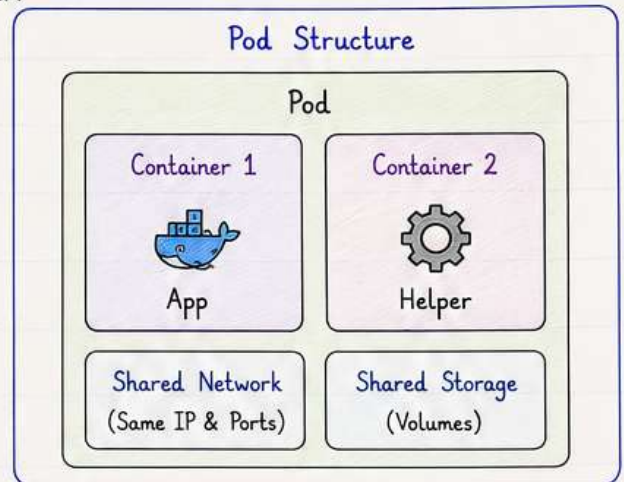
Kubernetes Handbook

3. Pods - The Smallest Deployable Unit

A **Pod** is the smallest and simplest Kubernetes object. It represents a single instance of a running process in your cluster.

Key Points

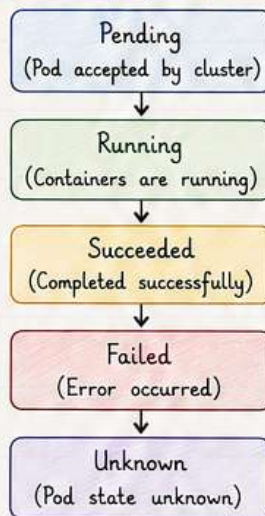
- A Pod can contain one or more containers.
- Containers in a Pod share the same network namespace (same IP and port space).
- They can also share storage volumes.
- Pods are **ephemeral** (temporary).
- When a Pod is deleted, it is not recreated by Kubernetes.



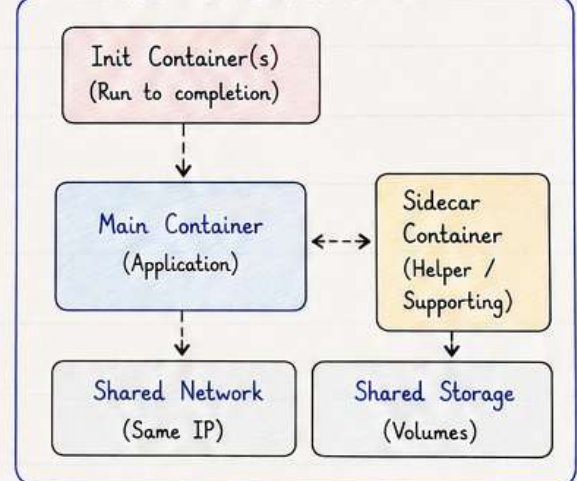
Types of Pods

- **Single Container Pod**
Contains only one container.
- **Multi Container Pod**
Contains multiple containers that work together.
- **Init Container**
Runs before the main containers start.
- **Sidecar Container**
Runs alongside the main container to extend its functionality.

Pod Lifecycle



Pod with Init & Sidecar



Common Pod Commands

Command	Description	Example
<code>kubectl get pods</code>	List all Pods in the namespace	<code>kubectl get pods</code>
<code>kubectl describe pod <pod-name></code>	Show detailed information about a Pod	<code>kubectl describe pod my-pod</code>
<code>kubectl logs <pod-name></code>	View logs of a container in a Pod	<code>kubectl logs my-pod</code>
<code>kubectl exec -it <pod-name> -- sh</code>	Open a shell inside the Pod	<code>kubectl exec -it my-pod -- sh</code>
<code>kubectl delete pod <pod-name></code>	Delete a Pod	<code>kubectl delete pod my-pod</code>
<code>kubectl get pods -o wide</code>	List Pods with more details	<code>kubectl get pods -o wide</code>
<code>kubectl get pods --watch</code>	Watch Pods in real-time	<code>kubectl get pods --watch</code>

Example

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
    - name: nginx
      image: nginx:1.25
      ports:
        - containerPort: 80
```



Interview Tip

- Pods are designed to be short-lived and replaceable.
- Use **Deployments** for managing Pods in production (not Pods directly).
- If a Pod fails, create a new Pod instead of fixing the old one.





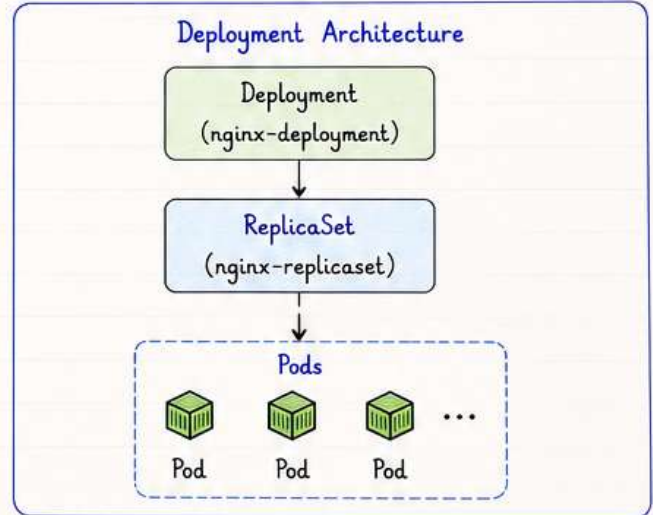
Kubernetes Handbook

4. Deployments - Declarative Updates for Pods

A **Deployment** manages **ReplicaSets** and ensures the desired number of Pod replicas are running. It provides declarative updates, rollbacks and scaling.

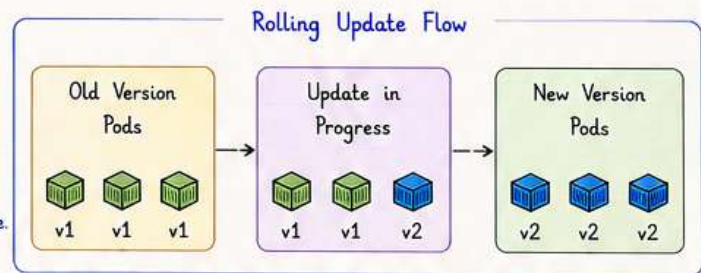
Key Concepts

- **Deployment**: Manages ReplicaSets and Pods.
- **ReplicaSet**: Ensures specified number of Pod replicas are running.
- **Desired State**: State defined in the Deployment manifest is the desired state.
- **Rolling Update**: Updates Pods gradually without downtime.
- **Rollback**: Reverts to a previous version easily.
- **Scaling**: Increase or decrease the number of Pod replicas.



How Deployment Works?

1. You define the desired state in a Deployment YAML.
2. Kubernetes creates/updates a ReplicaSet.
3. ReplicaSet creates the required number of Pods.
4. If a Pod fails, ReplicaSet replaces it (self-healing).
5. When you update the image, Deployment performs a Rolling Update.
6. Old Pods are gradually replaced with new Pods.
7. You can rollback to previous revision if needed.



No downtime! Traffic is always served by healthy Pods.

Common Deployment Commands

Command	Description	Example
<code>kubectl apply -f deployment.yaml</code>	Create or update a Deployment	<code>kubectl apply -f nginx-deploy.yaml</code>
<code>kubectl get deployments</code>	List all Deployments	<code>kubectl get deployments</code>
<code>kubectl describe deployment <name></code>	Show detailed info	<code>kubectl describe deployment nginx</code>
<code>kubectl rollout status deployment/<name></code>	Check rollout status	
<code>kubectl rollout history deployment/nginx</code>	View rollout history	<code>kubectl rollout history deployment/nginx</code>
<code>kubectl rollout undo deployment/<name></code>	Rollback to previous revision	<code>kubectl rollout undo deployment/nginx</code>
<code>kubectl scale deployment <name> --replicas=N</code>	Scale Deployment	<code>kubectl scale deployment nginx --replicas=5</code>

Example: Scale a Deployment

```
# Scale nginx deployment to 5 replicas
kubectl scale deployment nginx --replicas=5
# Verify
kubectl get pods
```

← All managed automatically by Kubernetes!

Deployment YAML (Minimal)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
```



Best Practice

- ✓ Always use Deployments for stateless applications.
- ✓ Set resource limits and requests.
- ✓ Use Rolling Update strategy (default) for zero downtime.
- ✓ Monitor rollout status and set proper readiness/liveness probes.





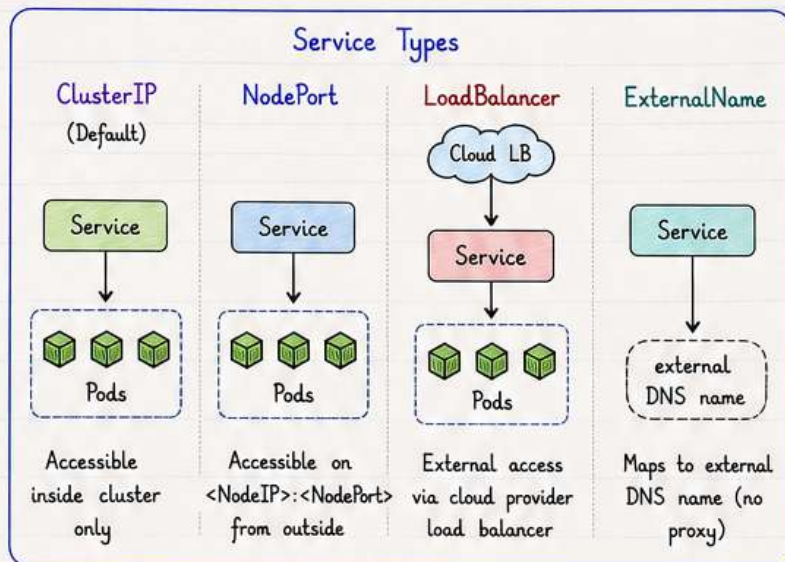
Kubernetes Handbook

5. Services & Networking

A **Service** is an abstraction which defines a logical set of **Pods** and a policy by which to access them.

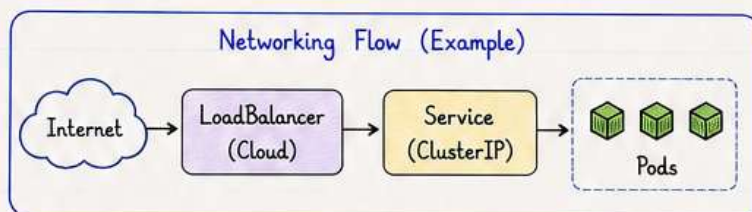
Key Concepts

- **Service**: Stable endpoint to access pods.
- **ClusterIP**: Default service type. Accessible only within the cluster.
- **NodePort**: Exposes service on each node's IP at a static port.
- **LoadBalancer**: Exposes service externally using cloud provider's load balancer.
- **ExternalName**: Maps service to an external DNS name.



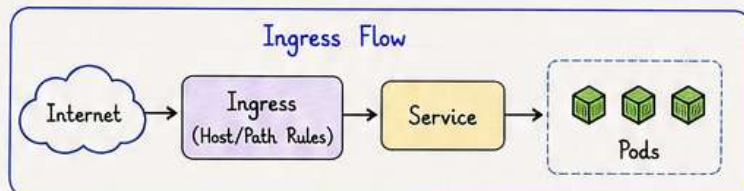
Service Discovery

- Kubernetes provides DNS for services.
- Format: `<service-name>.<namespace>.svc.cluster.local`
- Example: `my-service.default.svc.cluster.local`
- Pods can discover and communicate with each other using service names.



Ingress

- Ingress exposes HTTP/HTTPS routes from outside the cluster to services inside the cluster.
- It works at Layer 7 (Application Layer).
- Supports path-based & host-based routing, TLS termination, annotations, etc.



Common Service Commands

Command	Description	Example
<code>kubectl get svc</code>	List all services	<code>kubectl get svc</code>
<code>kubectl describe svc <name></code>	Show details of a service	<code>kubectl describe svc my-service</code>
<code>kubectl expose deployment <name> --port=80 --target-port=8080 --type=NodePort</code>	Create service for a deployment	<code>kubectl expose deployment nginx --port=80 --target-port=80 --type=NodePort</code>
<code>kubectl delete svc <name></code>	Delete a service	<code>kubectl delete svc my-service</code>

Service YAML (Example)

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 80      # Service port
      targetPort: 80 # Pod port
      nodePort: 30080 # NodePort (30000-32767)
```

Example: Accessing a NodePort Service

```
# Service is exposed on NodePort 30080
# Access from browser or curl
http://<NodeIP>:30080
```

← You can access the application from outside the cluster using any Node's IP and the allocated NodePort.



Interview Tip

- **ClusterIP** is used for internal communication between services.
- **NodePort** is used for development/testing.
- **LoadBalancer** is used in production to expose services externally.
- **Ingress** is preferred for HTTP/HTTPS routing and domain-based access.





Kubernetes Handbook

6. ConfigMaps & Secrets

Applications often need configuration data (like properties, URLs, feature flags) and sensitive data (like passwords, tokens). Kubernetes provides **ConfigMaps** and **Secrets** to handle this.

ConfigMap

- Stores **non-sensitive** configuration data as key-value pairs.
- Can be used as environment variables or mounted as files.
- Helps separate configuration from application code.

Example

```
kubectl create configmap app-config \
  --from-literal=APP_MODE=prod \
  --from-literal=LOG_LEVEL=info
```

Secret

- Stores **sensitive** data such as passwords, tokens, API keys, TLS certs.
- Data is stored in **base64** encoded format (not encrypted by default).
- Restrict access using RBAC.

Example

```
kubectl create secret generic db-secret \
  --from-literal=DB_USER=admin \
  --from-literal=DB_PASS=StrongPass123
```

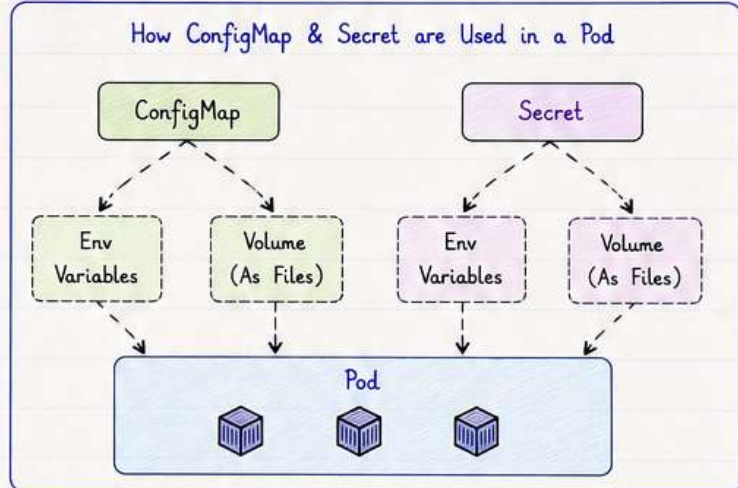
Ways to Use in Pods

- As **Environment Variables**
→ Inject key-value pairs into the container.
- As **Volume (Files)**
→ Mount as files inside the container.
- Using **envFrom / valueFrom**
→ Load multiple keys at once.

Mount ConfigMap as Volume (Example)

```
volumeMounts:
- name: config-volume
  mountPath: /etc/config
volumes:
- name: config-volume
  configMap:
    name: app-config
    items:
    - key: app.properties
      path: app.properties
```

Mounts the ConfigMap as a file inside the container.



ConfigMap vs Secret

Feature	ConfigMap	Secret
Purpose	Store non-sensitive data	Store sensitive data
Data Format	Plain text	Base64 encoded
Security	Not for sensitive info	Designed for sensitive info
Use Cases	App config, feature flags, URLs, properties	Passwords, tokens, API keys, TLS certs
Access Control	RBAC	RBAC (Recommended)
Encryption	No (By default)	Base64 only (not encrypted)

Pod Using ConfigMap & Secret (YAML Example)

```
apiVersion: v1
kind: Pod
metadata:
  name: my-app
spec:
  containers:
  - name: app
    image: nginx
    env:
    - name: APP_MODE
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: APP_MODE
    - name: DB_PASS
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: DB_PASS
```

From ConfigMap

From Secret

Important Notes

- ★ Secrets are encoded, not encrypted. Use encryption at rest (etcd encryption).
- ★ Limit access using RBAC.
- ★ Do not store secrets in images or code.
- ★ Use external secret managers (HashiCorp Vault, AWS Secrets Manager, etc.) for production.

Useful Commands

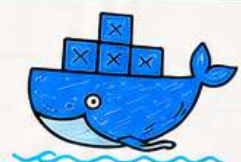
```
# ConfigMap
kubectl create configmap my-config
--from-file=app.properties
kubectl get configmaps

# Secret
kubectl create secret generic my-secret
--from-literal=key=value
kubectl get secrets
kubectl describe secret my-secret
```



Interview Tip

- Use ConfigMaps for configuration data, Secrets for secrets.
- Never hardcode secrets in images or manifests.
- In production, always enable etcd encryption and use RBAC.
- Prefer external secret stores for better security.





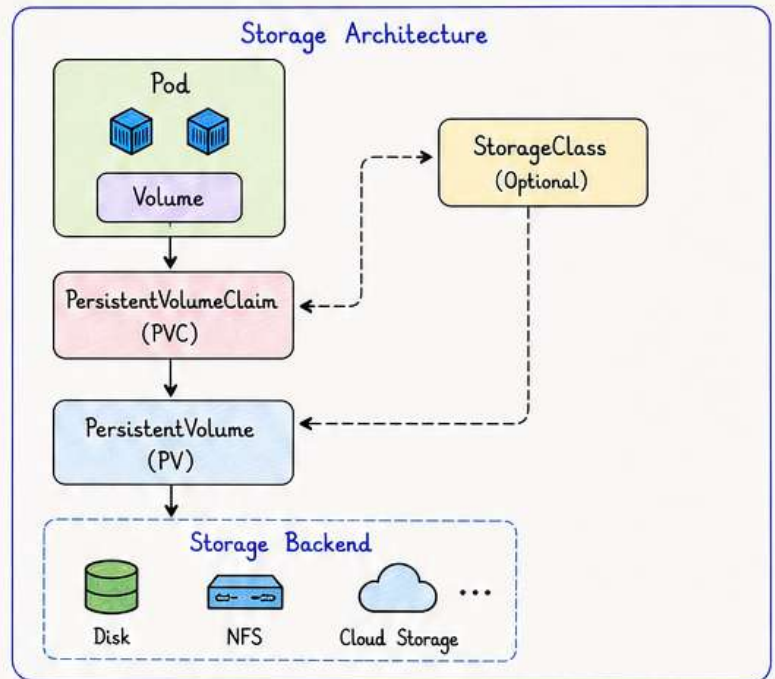
Kubernetes Handbook

7. Storage in Kubernetes

Kubernetes provides persistent storage using **Volumes** and **PersistentVolumes**.

Key Concepts

- **Volume**: Storage that is provisioned in a Pod and can be shared among containers in that Pod.
- **PersistentVolume (PV)**: A piece of storage in the cluster that has been provisioned by an administrator.
- **PersistentVolumeClaim (PVC)**: A request for storage by a user.
- **StorageClass**: Defines different types of storage (performance, replication, etc.).
- **Access Modes**:
 - ReadWriteOnce (RWO)
 - ReadOnlyMany (ROX)
 - ReadWriteMany (RWX)



Volume Types (Examples)

Type	Description
emptyDir	Temporary storage (deleted when Pod is removed)
hostPath	Mounts a file or directory from the node
configMap	Stores ConfigMap data as files
secret	Stores Secret data as files
persistentVolumeClaim	Uses storage provisioned via PVC
nfs	Mounts NFS share
csi	Uses CSI drivers for storage

PV <-> PVC Binding

1. User creates a PVC requesting storage.
2. Kubernetes finds a matching PV (based on size, access modes, StorageClass).
3. PVC is bound to the PV.
4. Pod uses the PVC and gets the storage from the bound PV.
5. When the Pod is deleted, the volume data persists.

Useful Commands

```
# List PVs
kubectl get pv

# List PVCs
kubectl get pvc

# Describe PV/PVC
kubectl describe pv pv-data
kubectl describe pvc pvc-data

# Check storage classes
kubectl get storageclass

# Delete PVC (data remains)
kubectl delete pvc pvc-data

# Delete PV (data may be lost)
kubectl delete pv pv-data
```

Example: Using a PVC in a Pod

1. PersistentVolume (PV)

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-data
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /data/pv
```

2. PersistentVolumeClaim (PVC)

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-data
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
```

3. Pod Using PVC

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
    - name: app
      image: nginx
      volumeMounts:
        - name: data
          mountPath: /usr/share/nginx/html
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: pvc-data
```

Best Practices

- ✓ Use PVCs instead of hostPath for portability.
- ✓ Choose the right access mode for your workload.
- ✓ Use StorageClass for dynamic provisioning.
- ✓ Back up important data.



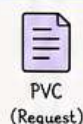
Interview Tip

- Volumes live a way for users to request storage without knowing the underlying storage details.
- PVs are cluster resources managed by admins.
- Separate application code from data.



Summary

Volumes live with the Pod.
PersistentVolumes live beyond the Pod.
PVC is the bridge that connects the two.





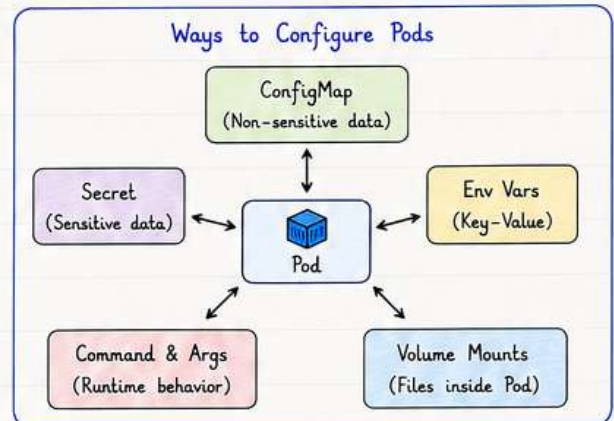
Kubernetes Handbook

8. Configuring Applications

Kubernetes provides several ways to configure applications without rebuilding container images.

Key Concepts

- **ConfigMap**: For non-sensitive configuration data.
- **Secret**: For sensitive data like passwords, tokens, certificates.
- **Environment Variables**: Simple key-value pairs.
- **Command & Args**: Override container startup behavior.
- **Volume Mounts**: Provide config files inside containers.



ConfigMap Example

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_MODE: "production"
  LOG_LEVEL: "info"
  WELCOME_MSG: "Hello from Kubernetes!"
```

← Data stored as key-value pairs. Can be mounted as env vars or files.

Secret Example

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  username: YWRtaW4= # base64 for 'admin'
  password: U3Ryb25nUGFzc2EyMw== # base64
```

← Encoded (base64) by default. Use type for specific secrets.

Using in a Pod Manifest (Example)

```
apiVersion: v1
kind: Pod
metadata:
  name: my-app
spec:
  containers:
    - name: app
      image: nginx
      envFrom:
        - configMapRef:
            name: app-config
      env:
        - name: DB_USER
          valueFrom:
            secretKeyRef:
              name: db-secret
              key: username
      volumeMounts:
        - name: config-volume
          mountPath: /app/config
      volumes:
        - name: config-volume
          configMap:
            name: app-config
            items:
              - key: WELCOME_MSG
                path: welcome.txt
```

← Load all keys from ConfigMap as environment variables.

← Load specific key from Secret as an environment variable.

← Mount ConfigMap data as files.

Config Methods Comparison

Method	Best For	How it Works	Example Use Case
Env Vars	Simple values	Set key-value in Pod spec	APP_MODE=prod
ConfigMap	Non-sensitive data	As env vars or files	App config, URLs
Secret	Sensitive data	As env vars or files	Passwords, tokens
Command & Args	Runtime control	Override container CMD	Debug mode, flags
Volume Mounts	Files / configs	Mount as files in Pod	Config files, scripts

Command & Args Example

```
containers:
- name: app
  image: my-app:latest
  command: ["/bin/sh", "-c"]
  args: ["python app.py --debug"]
```

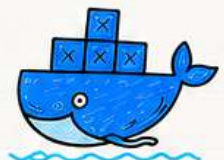
← Overrides the default command defined in the image.

Common Commands

Command	Description
<code>kubectl create configmap my-config</code>	Create a ConfigMap from literal values
<code>kubectl create secret generic my-secret</code>	Create a Secret from literal values
<code>kubectl get configmap</code>	List all ConfigMaps
<code>kubectl get secret</code>	List all Secrets
<code>kubectl describe configmap <name></code>	Show details of a ConfigMap
<code>kubectl describe secret <name></code>	Show details of a Secret

Best Practice

- Use ConfigMaps for application configuration.
- Use Secrets for sensitive data.
- Apply least privilege (RBAC).
- Avoid hardcoding configuration.
- Version control your ConfigMaps and manifests.



Interview Tip

- ConfigMaps and Secrets provide a clean separation between configuration and application.
- Secrets are not 100% secure by default—enable encryption and limit access.
- Understand how to inject configuration as environment variables or files.





Kubernetes Handbook

9. Configuring Applications (Advanced)

This section covers advanced configuration techniques for real-world applications.

Key Concepts

- **Resources:** CPU/Memory requests and limits.
- **Health Checks:** Liveness, Readiness, Startup probes.
- **ConfigMaps & Secrets:** Externalize configuration.
- **Downward API:** Expose Pod/Container metadata to applications.
- **Init Containers:** Run setup tasks before main containers.

Health Checks (Probes)

Probe	Purpose	When Used	Failure Result
Startup Probe	Checks if app is started	Slow starting apps	Container restarted if it fails
Liveness Probe	Checks if app is alive	All apps	Container restarted if it fails
Readiness Probe	Checks if app is ready to serve traffic	All apps	Pod removed from Service endpoints

Downward API Example

```
env:  
- name: POD_NAME  
  valueFrom:  
    fieldRef:  
      fieldPath: metadata.name  
- name: POD_IP  
  valueFrom:  
    fieldRef:  
      fieldPath: status.podIP
```

← Injects Pod metadata into environment variables.
Useful for logging, monitoring, etc.

Comprehensive Pod Example

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: advanced-app  
spec:  
  initContainers:  
    - name: init-config  
      image: busybox  
      command: ['sh', '-c', 'cp /config/* /app/config/']  
      volumeMounts:  
        - name: config-volume  
          mountPath: /config  
  containers:  
    - name: app  
      image: my-app:1.0  
      ports:  
        - containerPort: 8080  
      env:  
        - name: POD_NAME  
          valueFrom:  
            fieldRef:  
              fieldPath: metadata.name  
      resources:  
        requests:  
          cpu: "250m"  
          memory: "256Mi"  
        limits:  
          cpu: "500m"  
          memory: "512Mi"  
      readinessProbe:  
        httpGet:  
          path: /ready  
          port: 8080  
        initialDelaySeconds: 5  
        periodSeconds: 10  
      livenessProbe:  
        httpGet:  
          path: /healthz  
          port: 8080  
        initialDelaySeconds: 10  
        periodSeconds: 20  
      volumeMounts:  
        - name: config-volume  
          mountPath: /app/config  
  volumes:  
    - name: config-volume  
      configMap:  
        name: app-config
```

Pod Resource Management

Requests

Minimum resources guaranteed to the Pod.



Limits

Maximum resources a Pod can use.



Always set both requests and limits for predictable performance and scheduling.

Probe Types

- **httpGet:** Perform an HTTP GET request.
- **tcpSocket:** Check if TCP port is open.
- **exec:** Execute a command in the container.

Example (Readiness Probe - httpGet)

```
readinessProbe:  
  httpGet:  
    path: /ready  
    port: 8080  
  initialDelaySeconds: 5  
  periodSeconds: 10  
  timeoutSeconds: 2  
  failureThreshold: 3  
  successThreshold: 1
```

Init Containers Example

```
initContainers:  
- name: init-db  
  image: busybox  
  command: ['sh', '-c', 'until nslookup db; do echo waiting for db; sleep 2; done']  
  
containers:  
- name: app  
  image: my-app:latest
```

← Waits for the database to be available before starting the main container.

Resources Best Practices

- ✓ Set requests and limits based on application needs.
- ✓ Avoid requesting more than necessary.
- ✓ Monitor usage and adjust over time.
- ✓ Use Vertical Pod Autoscaler (VPA) for recommendations.
- ✓ Do not set CPU limits too low (causes throttling).

Common Mistakes

- ✗ Not setting resource requests/limits.
- ✗ Using same ConfigMap for sensitive and non-sensitive data.
- ✗ Forgetting liveness probe (causes stale pods).
- ✗ Mounting secrets as environment variables in logs.
- ✗ Not testing probes under failure conditions.

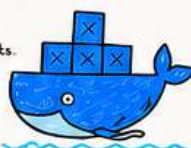
Useful Commands

```
# Describe a Pod in detail  
kubectl describe pod <pod-name>  
# Get Pod resource usage  
kubectl top pod <pod-name>  
# Get events related to a Pod  
kubectl get events --field-selector involvedObject.name=<pod-name>
```



Best Practice

- Externalize configuration using ConfigMaps and Secrets.
- Use probes to improve reliability and availability.
- Keep containers stateless and idempotent.
- Automate configuration using Helm or Kustomize.



Interview Tip

- Explain the difference between liveness and readiness probes.
- Know use cases of init containers and downward API.
- Understand why resource limits are important.
- Be ready to write YAML for advanced Pod specs.



Kubernetes Handbook

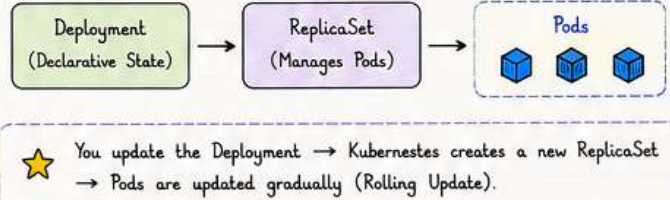
10. Deployments & Updates

Deployments provide declarative updates for Pods and ReplicaSets.

Key Concepts

- Deployment:** Manages ReplicaSets and provides declarative updates.
- ReplicaSet:** Ensures the desired number of Pod replicas.
- Rolling Update:** Gradual update with zero downtime.
- Rollback:** Roll back to a previous version.
- Strategies:** RollingUpdate (default) or Recreate.

How a Deployment Works



Deployment YAML (Example)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  labels:
    app: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: nginx
          image: nginx:1.25-alpine
          ports:
            - containerPort: 80
```

← Name of the Deployment

← Number of Pod replicas

← Select Pods with label app=my-app

← Pod template that ReplicaSet will use

Update Strategies

Strategy	Description	When to Use
RollingUpdate (Default)	Gradually replaces old Pods with new Pods. Ensures zero downtime.	Recommended for most applications.
Recreate	Terminates all old Pods before creating new ones.	For applications that cannot run multiple versions.

Rolling Update Process



Common Deployment Commands

Command	Description	Example
<code>kubectl create deployment</code>	Create a deployment	<code>kubectl create deployment my-app --image=nginx</code>
<code>kubectl get deployments</code>	List deployments	<code>kubectl get deployments</code>
<code>kubectl describe deployment <name></code>	Show details of a deployment	<code>kubectl describe deployment my-app</code>
<code>kubectl rollout status deployment/<name></code>	Check rollout status	<code>kubectl rollout status deployment/my-app</code>
<code>kubectl set image deployment/<name> <container>=<image></code>	Update image of a deployment	<code>kubectl set image deployment/my-app nginx=nginx:1.26</code>
<code>kubectl rollout history deployment/<name></code>	Show rollout history	<code>kubectl rollout history deployment/my-app</code>
<code>kubectl rollout undo deployment/<name></code>	Undo to previous revision	<code>kubectl rollout undo deployment/my-app</code>

Rollback Example

```
# Check rollout history
kubectl rollout history deployment/my-app

# Rollback to previous revision
kubectl rollout undo deployment/my-app

# Rollback to a specific revision
kubectl rollout undo deployment/my-app --to-revision=2
```

← Reverts the Deployment to a previous ReplicaSet.

Deployment Status Conditions

Condition	Meaning
Available	Deployment has the minimum required Pods available.
Progressing	Deployment is making progress.
ReplicaFailure	Deployment failed to create or delete Pods.



Best Practices

- ✓ Use RollingUpdate strategy for most workloads.
- ✓ Always set resource requests and limits.
- ✓ Use liveness and readiness probes.
- ✓ Keep deployments stateless when possible.
- ✓ Use labels consistently for selectors.



Common Mistakes

- ✗ Using Recreate strategy for production workloads.
- ✗ Forgetting to set resource limits.
- ✗ Not checking rollout status after updates.
- ✗ Deleting deployments instead of rolling back.
- ✗ Using same image tag (e.g., latest) without version control.



Interview Tip

- Be ready to explain how rolling updates work.
- Know how to rollback a deployment.
- Understand ReplicaSets and how they are managed by Deployments.
- Practice common rollout commands.

